

AI Career Intelligence & Application Platform

End-to-End AI-Powered Job Discovery, Matching, Application Personalization & Tracking System

1. Executive Summary

The AI Career Intelligence & Application Platform is a personal AI career operating system that continuously discovers, evaluates, ranks, prepares, and tracks job opportunities. It centralizes the repetitive work of job searching while keeping the final decision and application submission under the user's control.

The revised system is no longer centered on a large collection of rigid preference filters. Instead, the user can describe their career goal in natural language—for example: “Data professional jobs in Germany or the UK, preferably fully remote, 3+ years experience, with visa sponsorship.” The system converts this statement into structured, editable search constraints such as target roles, locations, seniority, work mode, salary, sponsorship, exclusions, and other criteria.

The application-generation layer is also reframed. Rather than simply rewriting a resume into a new template, the system optimizes the user's master resume for a specific job by identifying missing or weakly represented keywords and phrases, then naturally incorporating only facts supported by the candidate profile. Each generated resume is evaluated for keyword coverage, semantic alignment, factual consistency, and machine readability.

The platform is designed for personal open-source portability. The default local development environment runs through Docker Compose, PostgreSQL/pgvector, MinIO, and application workers. Cloudflare R2 remains an optional production object-storage backend rather than a mandatory dependency. Background jobs are handled by BullMQ and scheduled jobs by a lightweight cron/Vercel Cron strategy, avoiding a hard dependency on n8n Cloud.

Finally, the platform learns from application outcomes. Application history—job characteristics, match scores, resume optimization scores, and outcomes such as interviews or rejections—feeds an initial analytics/learning layer that can later predict which jobs are most promising for the individual user.

2. Problem Statement

Traditional job searching is repetitive and fragmented. A candidate must search multiple sources, inspect job descriptions, judge eligibility, compare requirements with their resume, check location and sponsorship, save jobs, customize applications, and track outcomes.

- Search multiple job boards and company career pages.
- Determine whether a role actually fits the candidate's profile and constraints.
- Compare job requirements against skills, experience, projects, and achievements.
- Check location, work mode, salary, experience, and visa/work-authorization requirements.
- Prepare job-specific application materials.
- Track applications, follow-ups, interviews, rejections, and offers.
- Learn which types of roles and application strategies actually produce interviews.

The revised objective is not to remove the human from the loop. It is to compress the repetitive research and preparation work so the user can make better decisions and apply faster.

3. Product Objective

The system should answer a stronger version of the original product question:

“Given my career profile and natural-language career goal, which jobs should I consider, why are they relevant, what requirements are missing or risky, and can the system prepare a factual, ATS-readable application package for the jobs I select?”

- Understand the user's career profile and master resume.
- Convert a natural-language career goal into structured, editable constraints.

Continuously discover and normalize relevant jobs.

- Use deterministic eligibility filtering before expensive AI processing.
- Use hybrid lexical + semantic retrieval to avoid missing exact requirements.
- Rank jobs using personalized scoring and explain the ranking.
- Prepare an ATS-oriented resume variant without fabricating experience.

• Provide a machine-readability/ATS confidence report with measurable sub-scores. • Generate a concise Hiring Manager Pitch instead of relying primarily on generic cover letters. • Autofill supported application forms and stop before final submission.

- Track application outcomes and use them to improve future recommendations. **4.**

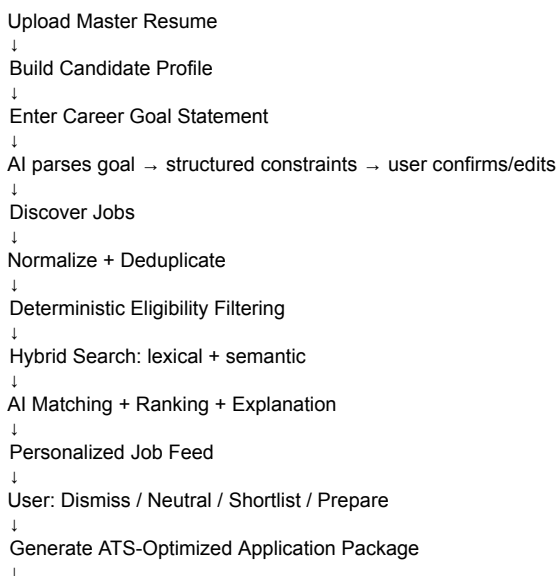
Product Philosophy

AI recommends. The user decides.

- The platform must not autonomously submit job applications.
- Browser automation may navigate and autofill supported forms, but it must stop before the final Submit/Apply action.
- The candidate profile is the factual source of truth for generated application materials.
- Every AI transformation must be traceable to source facts and job requirements.
- The database is the source of truth for job and application state.
- Core business logic lives in application code; workers/orchestration execute jobs rather than defining domain rules.

5. Revised High-Level User Journey

The revised journey is:



ATS / Machine Readability Evaluation
↓
User Review
↓
Open Application Portal + Autofill Supported Fields
↓
STOP BEFORE FINAL SUBMISSION
↓
User Submits Manually
↓
Application Tracker + Outcome Feedback
↓
Better Future Recommendations

6. Core Features — Revised

6.1 Candidate Profile

- Education, work experience, skills, projects, certifications, achievements.
- Preferred roles, industries, countries/cities, work mode, salary expectations.
- Experience level and visa/work-authorization requirements.
- Preferred companies and excluded companies.
- Verified source facts used by resume and pitch generation.

6.2 Career Goal Statement — Replace Rigid Preference UI

The primary search input should be a natural-language Career Goal Statement. The system parses it into a structured object rather than forcing the user to configure many individual filters.

Example: “Data professional jobs in Germany/UK with visa sponsorship, 3+ YOE, fully remote, minimum €60k, excluding consulting.”

The parser should extract and normalize fields such as:

- Target roles and role families.
- Locations and geographic scope.
- Remote/hybrid/onsite preference.
- Minimum/target experience.
- Salary floor and preferred compensation.
- Visa sponsorship / work authorization requirements.
- Industries and company preferences.
- Excluded companies or industries.
- Priority terms and hard constraints.

The generated structured goal must be shown to the user for confirmation and editing. Natural language is the input convenience layer; the structured object remains the deterministic matching contract.

7. Job Intelligence

Jobs may be collected from job APIs, company career pages, RSS/public feeds, Apify actors, and other legally/technically appropriate sources.

Each job is normalized into a persistent schema containing ID, company, title, description, location, remote status, employment type, salary, experience, required skills, preferred skills, responsibilities, qualifications, sponsorship, source, URL, posted date, and expiration date.

Deduplication should combine source job ID, company, normalized title, location, URL, description hash, and

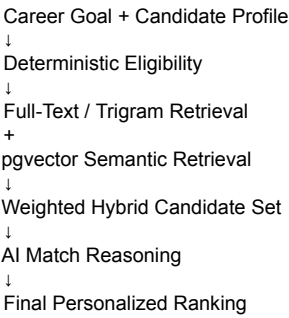
normalized content to create a persistent job identity.

8. Eligibility + Hybrid Matching Engine

Before expensive AI analysis, deterministic rules remove clearly incompatible roles. Examples include disallowed countries, already-applied jobs, permanently dismissed jobs, hard experience mismatches, and onsite roles when remote is a hard requirement.

Hybrid retrieval is mandatory: combine exact/lexical matching with semantic similarity. PostgreSQL full-text search and pg_trgm handle exact terms and fuzzy lexical overlap, while pgvector handles semantic similarity. This prevents vector-only retrieval from missing rare but decisive requirements such as a specific certification, technology, or tool.

A practical ranking pipeline is:



Initial ranking factors can retain the original structure while allowing future learning:

Factor	Initial Weight	Notes
Skills / requirements	30%	Exact + semantic alignment
Experience	15%	Seniority and years
Location / work mode	15%	Hard constraints first
Sponsorship	10%	Critical when required
Role preference	10%	Career-goal alignment
Salary	5%	Floor and preference
Industry / company	5%	Goal alignment
Job freshness	5%	Prefer newer postings
Semantic fit	5%	Overall contextual fit

9. Recommendation System

Every recommendation should provide both a score and an explanation. Example reasons include strong SQL/Python alignment, matching work mode, acceptable experience, sponsorship availability, and specific gaps such as Tableau being requested while the candidate profile primarily demonstrates Power BI.

Add a Job Posting Age Score: boost recent roles (for example, under 24 hours), apply a smaller boost to recent roles, and progressively reduce ranking for stale postings. The exact decay should be configurable and treated as a ranking signal rather than a guarantee of hiring probability.

Add negative preferences as first-class structured data. The user can state “Exclude: Accenture, Deloitte, Amazon” or exclude industries/company categories. These exclusions should be deterministic rather than relying only on prompt wording.

10. Application Generation — New Model

The application package should prioritize factual, ATS-oriented optimization over cosmetic rewriting. For each

selected job, the system creates:

- ATS-optimized resume variant.
- Machine Readability / ATS Confidence report.
- Three-bullet Hiring Manager Pitch.
- Company research summary.
- Interview preparation.
- Optional legacy cover letter mode, but cover letters are secondary rather than the core value proposition.

10.1 ATS Keyword Injection & Semantic Weighting

The master resume remains the canonical source. Given a job description, the system extracts required and preferred terms, identifies missing or underweighted terms, and naturally strengthens relevant existing bullets without inventing facts.

The objective is not to guarantee a particular ATS result. The system should instead optimize measurable coverage and readability against the target job and report the result transparently.

The optimizer may:

- Reorder relevant bullets.
- Rewrite phrasing while preserving factual meaning.
- Use job-relevant terminology when it is supported by the candidate's actual experience. • Increase semantic prominence of genuinely relevant skills.
- Surface relevant projects or achievements.
- Never add unsupported skills, employers, metrics, titles, certifications, or experience.

10.2 Machine Readability / ATS Confidence

Before export, evaluate the generated resume against the job description using a measurable scorecard. • Keyword coverage.

- Required-skill coverage.
- Semantic similarity to the job requirements.
- Action-verb quality and bullet clarity.
- Section/format machine readability.
- Experience alignment.
- Factual consistency.
- Potential hallucination or unsupported-claim risk.

Example report: 92/100 ATS-oriented match, with 94% required-keyword coverage, 89% semantic alignment, 100% factual consistency, and 95% machine-readable formatting. A configurable target such as 95/100 may be used as an optimization threshold, but the system must never falsely claim that a score guarantees passing an external ATS.

11. Structured Outputs & Hallucination Guardrails

All important LLM workflows should use structured outputs validated with Zod. The resume optimizer should return explicit changes and their factual justification.

Example schema:

```
{
  "modified_bullet_points": [...],
  "added_terms": [...],
  "justifications": [...],
  "unsupported_claims_detected": [...],
  "requires_review": true
}
```

A rewrite should be rejected when the model cannot connect the proposed change to verified candidate information or a specific job requirement. This creates an auditable chain from job requirement → source fact → generated wording.

12. Hiring Manager Pitch — Replace Generic Cover Letter as Primary Output

The primary communication artifact should be a concise three-bullet Hiring Manager Pitch: • Why this company?

- Why this role?
- Why this candidate?

The pitch should be grounded in company research and verified candidate facts. A traditional cover letter can remain an optional secondary feature for applications that explicitly request one.

13. Application Automation — Human-in-the-Loop

Add a browser-automation worker using Playwright. The worker can use the stored candidate profile to autofill supported application forms on common ATS platforms such as Workday, Lever, and Greenhouse where technically feasible.

The workflow is intentionally not a fully autonomous application bot:

```
Prepare
↓
Generate Resume + Pitch
↓
Open Application Portal
↓
Detect Supported Form
↓
Autofill Candidate Fields
↓
Attach Generated Resume
↓
Pause for Human Review
↓
USER CLICKS FINAL SUBMIT
```

The automation must stop before the final submission action. It should log which fields were filled and flag fields that require manual attention. This preserves user control and reduces the risk of incorrect or unauthorized submissions.

14. Application Tracker & Feedback Loop

The application tracker remains the central dashboard and should store company, job, application date, resume version, pitch/cover letter, job URL, status, recruiter, interview dates, salary, notes, follow-up date, and application events.

Extend the tracker with outcome-learning fields:

- Job type / role family.
- Company and industry.
- Location and work mode.
- Salary range.
- Match score.
- ATS-oriented score.

- Keyword coverage.
- Resume version.
- Pitch version.
- Application date / posting age.
- Interview outcome.
- Rejection / offer outcome.

After sufficient history, train a simple personal model such as logistic regression to estimate interview probability from historical application features. Start with interpretable statistics and only introduce a more complex model if the dataset supports it. The system should show confidence and sample size rather than pretending that a small personal dataset produces a reliable universal prediction.

15. Salary & Job Freshness Intelligence

Extract salary information when available and compare it with the user's configured salary floor. Jobs below the floor can be marked Low Priority unless explicitly overridden.

Salary extraction must distinguish between exact values, ranges, currencies, annual/monthly compensation, and missing information. Missing salary should not be treated as zero.

Job age should influence ranking. Fresh postings receive a configurable boost, while older postings decay in priority. The system should retain the original posted date and the date last verified.

16. Rejection & Re-optimization Loop

When an application is rejected, the tracker can ask whether the application used the platform's optimized resume and pitch.

If yes, record the outcome against the exact job description, resume version, ATS score, and match features. The system can identify patterns such as weak performance for a role family, recurring missing requirements, or poor outcomes despite high lexical coverage.

This should improve future ranking and optimization strategies rather than repeatedly rewriting the same resume blindly.

17. Technology Architecture — Open-Source / Portable First

The original architecture used Vercel, Supabase, Cloudflare R2, and n8n Cloud. The revised architecture keeps these services compatible but removes unnecessary mandatory vendor dependencies.

Recommended deployment modes:

Component	Local / OSS Default	Optional Cloud
Web	Next.js + TypeScript	Vercel
Database	PostgreSQL	Supabase PostgreSQL

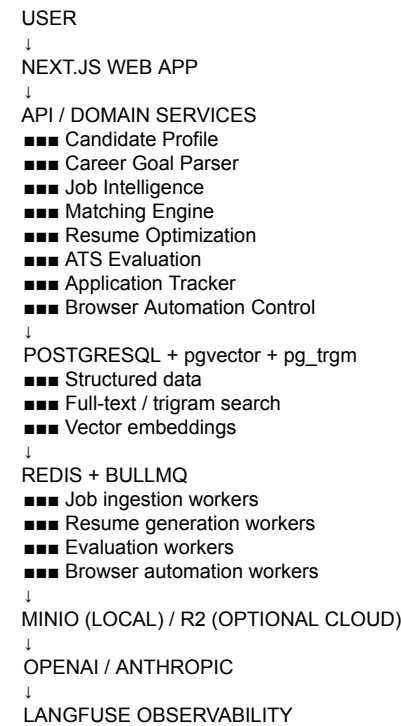
Vector Search	pgvector	Supabase pgvector
Lexical Search	PostgreSQL FTS + pg_trgm	Same
Object Storage	MinIO	Cloudflare R2
Background Jobs	BullMQ + Redis	Managed Redis
Scheduling	node-cron / Vercel Cron	Vercel Cron

Browser Automation	Playwright worker	Container/worker service
AI	OpenAI / Anthropic adapters	Same
Observability	Langfuse	Langfuse Cloud or self-hosted
Secrets	.env locally	Doppler optional

Docker Compose is a first-class deliverable. A fresh developer should be able to clone the repository, configure environment variables, run docker compose, initialize PostgreSQL/pgvector + Redis + MinIO, and start the application locally without requiring paid SaaS services.

n8n can remain an optional integration for users who want visual workflows, but core application logic and background execution should not depend on n8n Cloud.

18. Revised System Architecture



19. Database Additions

Retain the original core tables—users, profiles, resumes, resume_versions, skills, candidate experiences/projects, preferences, companies, jobs, job sources, job skills, job matches, applications, application events, saved/dismissed jobs, documents, generated resumes, cover letters, interview preparations, and notifications.

Add or extend tables for the revised features:

- career_goals — natural-language goal, parsed structured goal, version, confirmation status.
- career_goal_constraints — normalized roles, locations, work mode, salary, experience, sponsorship, exclusions, and priorities.
- job_requirements — normalized required/preferred terms and requirement types.
- resume_optimizations — job ID, resume version, changes, added terms, justifications, evaluation scores.
- ats_evaluations — keyword coverage, semantic score, factuality, machine readability, overall score, evaluator version.
- application_pitches — three-bullet pitch versions and source evidence.

- automation_sessions — portal, detected fields, filled fields, review state, stopped-before-submit flag.
- application_outcomes — interview/rejection/offer signals and timestamps.
- learning_features — historical feature snapshots used for personal outcome prediction.

20. Revised Repository Structure

```

ai-career-intelligence/
  ■■■ apps/
  ■ ■■■ web/
  ■■■ packages/
  ■ ■■■ db/
  ■ ■■■ ai/
  ■ ■■■ matching/
  ■ ■■■ resume/
  ■ ■■■ evaluation/
  ■ ■■■ browser/
  ■■■ services/
  ■ ■■■ job-ingestion/
  ■ ■■■ resume-generation/
  ■ ■■■ matching-worker/
  ■ ■■■ evaluation-worker/
  ■ ■■■ browser-worker/
  ■■■ workflows/
  ■ ■■■ cron/
  ■ ■■■ optional-n8n/
  ■■■ infra/
  ■ ■■■ docker/
  ■ ■■■ docker-compose.yml
  ■ ■■■ minio/
  ■■■ tests/
  ■ ■■■ unit/
  ■ ■■■ integration/
  ■ ■■■ e2e/
  ■■■ docs/
  ■ ■■■ architecture/
  ■ ■■■ database/
  ■ ■■■ api/
  ■ ■■■ decisions/
  ■■■ scripts/
  ■■■ .github/workflows/
  ■■■ package.json
  ■■■ turbo.json
  ■■■ README.md

```

21. Revised Development Roadmap

Phase 0 — Technology Learning

Learn each technology through small experiments that later become project components. **Phase 1**

— Foundation

GitHub, Turborepo, Next.js, PostgreSQL/pgvector, Redis, Docker Compose, MinIO, authentication, environment configuration, CI/CD.

Phase 2 — Candidate Intelligence

Resume upload/parsing, candidate profile extraction, manual editing, resume versioning, source-of-truth facts.

Phase 3 — Career Goal Intelligence

Natural-language Career Goal Statement parser, structured constraints, user confirmation/editing, excluded companies, salary floor.

Phase 4 — Job Intelligence

Job sources, ingestion workers, normalization, deduplication, posted-date tracking, salary extraction.

Phase 5 — Hybrid Matching

Deterministic eligibility, PostgreSQL FTS/pg_trgm, pgvector, hybrid scoring, ranking, match explanations, job freshness.

Phase 6 — ATS Resume Optimization

Requirement extraction, keyword injection, semantic weighting, structured outputs, factuality checks, ATS/machine-readability evaluator, iterative improvement.

Phase 7 — Application Package

Three-bullet Hiring Manager Pitch, company research, interview preparation, optional cover letter, document storage.

Phase 8 — Browser Automation

Playwright worker, supported ATS adapters, profile autofill, resume attachment, field audit, mandatory stop-before-submit behavior.

Phase 9 — Application Tracker

Lifecycle state, resume/pitch association, follow-ups, recruiter/interview events, outcomes. **Phase**

10 — Feedback & Learning

Interview/rejection analytics, application outcome features, interpretable baseline model, personal ranking adjustments.

Phase 11 — Observability & Production Hardening

Langfuse, rate limiting, retries, security, indexes, automated testing, E2E tests, cost controls, monitoring, documentation.

22. Revised MVP Definition

The first genuinely useful version should focus on the smallest end-to-end loop that demonstrates the project's unique value:

```
Upload Resume
↓
Enter Career Goal Statement
↓
Confirm Structured Goal
↓
Find + Normalize Jobs
↓
Eligibility + Hybrid Search
↓
Rank Top 10
↓
User Shortlists
↓
Generate ATS-Optimized Resume
↓
Generate ATS/Machine Readability Report
↓
Generate Hiring Manager Pitch
↓
Open Application + Autofill
↓
STOP BEFORE SUBMIT
↓
Track Outcome
```

Browser automation, personal predictive modeling, company research depth, and advanced analytics can be phased in after the core loop is reliable.

23. Key Engineering Principles

¹ Database is the source of truth for candidate, job, and application state.

2 Natural language is a convenience layer; structured career-goal constraints drive deterministic matching.

3 Use deterministic filtering before semantic and LLM processing.

4 Hybrid lexical + semantic retrieval is required; vector search alone is insufficient. 5 AI outputs must be structured and schema-validated.

6 Never hallucinate candidate information.

7 Resume optimization must be evidence-backed and auditable.

8 ATS scores are internal evaluation signals, not guarantees of external ATS behavior. 9 Browser automation must stop before final application submission.

10 User remains in control of all final application decisions.

11 Every job has a persistent identity to prevent duplicates.

12 Every important AI workflow should be observable.

13 Prefer local/OSS-compatible infrastructure and keep cloud services optional where practical. 14 Measure outcomes and improve recommendations using actual application feedback.

24. Revised Success Criteria

1 Upload a resume and automatically generate a structured candidate profile. 2 Describe a career goal in natural language and receive an editable structured representation. 3 Discover jobs from multiple sources and deduplicate them.

4 Filter incompatible roles before expensive AI calls.

5 Retrieve jobs using both exact/lexical and semantic matching.

6 Receive a ranked list with transparent explanations.

7 Prioritize recent jobs and flag salary below the configured floor.

8 Dismiss jobs permanently and prevent already-applied jobs from reappearing. 9 Generate job-specific resumes grounded in verified candidate facts.

10 Measure keyword coverage, semantic alignment, factual consistency, and machine readability. 11 Generate a concise Hiring Manager Pitch.

12 Autofill supported application forms and stop before final submission.

13 Track applications, interviews, rejections, offers, and follow-ups.

14 Record application outcomes against resume versions and job features.

- 15 Use accumulated outcomes to improve future recommendations.
- 16 Run the complete system locally through Docker Compose without requiring paid SaaS services.

25. Final Product Definition

The completed platform should feel like a personal AI career operating system rather than a job board. The core experience becomes:

```
CAREER PROFILE
↓
NATURAL-LANGUAGE CAREER GOAL
↓
JOB INTELLIGENCE
↓
HYBRID SEARCH + AI MATCHING
↓
PERSONALIZED JOB FEED
↓
USER DECISION
↓
ATS RESUME OPTIMIZATION
↓
MACHINE READABILITY / ATS EVALUATION
↓
HIRING MANAGER PITCH
↓
HUMAN-IN-THE-LOOP APPLICATION AUTOMATION
↓
APPLICATION TRACKER
↓
CONTINUOUS FEEDBACK
↓
BETTER RECOMMENDATIONS
```

The revised project therefore demonstrates more than an LLM wrapper: it combines natural-language intent parsing, deterministic systems, hybrid information retrieval, vector search, structured generation, evaluation, browser automation, background workers, observability, local-first infrastructure, and outcome-based learning.

26. Final Technology Stack

Next.js + TypeScript + Tailwind + shadcn/ui + PostgreSQL + pgvector + pg_trgm + Redis + BullMQ + MinIO + Docker Compose + Playwright + OpenAI/Anthropic + Langfuse + Drizzle + Zod + GitHub + GitHub Actions + Turborepo + Vitest + Playwright

Optional cloud integrations: Vercel, Supabase, Cloudflare R2, Doppler, managed Redis, Langfuse Cloud, and optional n8n.

This revised specification supersedes the original project specification where the two differ, while retaining the original core product idea and its human-in-the-loop philosophy.